

리눅스 3.3 - B-Tree

남궁명수

[B-Tree]

자료구조:

- 하나의 노드에 여러 키와 여러 자식을 저장하는 균형 탐색 트리

키의 역할:

- 탐색할 자식 범위를 결정
- B-Tree에서는 실제 데이터 정보와 연결될 수도 있음

핵심 동작:

- 탐색: 키를 비교해 적절한 자식으로 이동
- 삽입: 노드가 가득 차면 분할
- 삭제: 키가 부족하면 빌리거나 병합

시간 복잡도:

- 탐색: $O(\log n)$
- 삽입: $O(\log n)$
- 삭제: $O(\log n)$

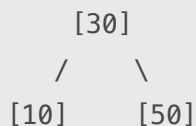
목적:

- 트리 높이를 낮게 유지
- 디스크 페이지 한 번에 여러 키 확인
- DB와 파일 시스템의 디스크 I/O 감소

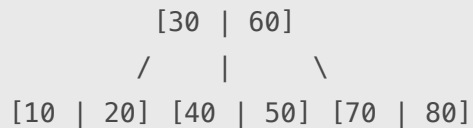
B-Tree는 하나의 노드에 여러 개의 키와 자식 포인터를 저장하는 균형 탐색 트리이다.

일반 이진 탐색 트리는 노드마다 키 하나와 최대 두 개의 자식만을 가진다.

이진 탐색 트리



B-Tree는 하나의 노드가 여러 키와 여러 자식을 가질 수 있다.



핵심

트리의 높이를 낮춰 디스크 접근 횟수를 줄이는 것

[전체 흐름]

B+Tree에서 키 탐색



레코드가 저장된 데이터 페이지 확인



해당 페이지가 RAM에 없으면 디스크에서 읽음



버퍼 풀에 올림



원하는 실제 레코드 반환

[1] 노드에 저장되는 정보

[키 | 키 | 키 ...]

[자식 포인터 | 자식 포인터 | 자식 포인터 ...]

예를 들어, 다음 노드가 있다고 하자.

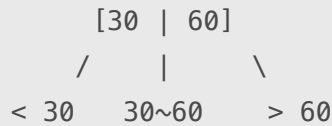
[30 | 60]

키 30, 60은 실제 데이터를 찾는 과정에서 어느 자식으로 내려갈지 결정하는 경계값이다.

30보다 작음 → 첫 번째 자식

30보다 크고 60보다 작음 → 두 번째 자식

60보다 큼 → 세 번째 자식



따라서 키가 n개라면 자식은 최대 n+1개다.

[2] B-Tree에서 키의 역할

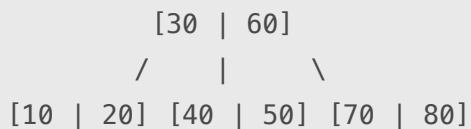
B-Tree의 키는 단순한 안내판만은 아니다.

일반적인 B-Tree에서는 내부 노드의 키도 실제 레코드 또는 실제 레코드를 가리키는 정보를 가질 수 있다.

내부 노드의 키

- └─ 탐색 방향을 결정하는 기준
- └─ 실제 데이터 또는 레코드 위치 정보와 연결될 수 있음

예를 들어, 50을 찾는다면



1. 루트에서 $30 < 50 < 60$ 확인
2. 가운데 자식으로 이동
3. $[40 | 50]$ 에서 50 발견
4. 연결된 실제 레코드에 접근

B-Tree에서는 탐색이 리프 노드까지 가지 않고 내부 노드에서 끝날 수도 있다.

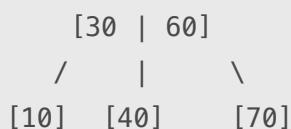
[3] B-Tree의 규칙

[키는 정렬되어 있다]

$[10 | 30 | 50]$

하나의 노드 안에서도 키가 오름차순으로 정렬되어 있다.

[모든 리프 노드의 높이가 같다]



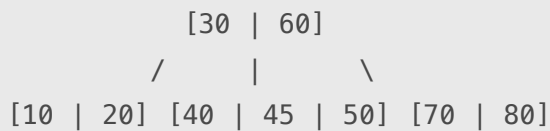
모든 리프 노드가 같은 깊이에 있으므로 B-Tree는 항상 균형을 유지한다.
따라서 한쪽으로 치우쳐 탐색 성능이 $O(n)$ 까지 떨어지는 문제가 발생하지 않는다.

[노드가 저장할 수 있는 키 개수가 정해져 있다]

각 노드는 정해진 범위만큼의 키를 저장해야 한다. 노드가 가득 찬 상태에서 새 키를 삽입하면 노드를 분할한다.

[4] 탐색 과정

B-Tree에서 45를 찾는다고 가정한다.



1. 루트에서 45와 30, 60 비교
2. $30 < 45 < 60$
3. 가운데 자식으로 이동
4. [40 | 45 | 50]에서 45 발견

노드 안의 키를 비교하고, 적절한 자식 노드로 내려가는 과정을 반복한다.

시간 복잡도는 다음과 같다.

탐색: $O(\log n)$

[5] 삽입과 노드 분할

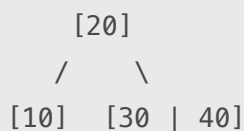
다음 리프 노드가 저장할 수 있는 키가 최대 3개라고 가정하자.

[10 | 20 | 30]

여기에 40을 삽입하면 최대 개수를 초과한다.

[10 | 20 | 30 | 40] ← Overflow

중간 키를 부모로 올리고 노드를 둘로 나눈다.



이 과정을 노드 분할(Split)이라고 한다.

부모 노드도 가득 차 있다면 분할이 위쪽으로 연속해서 전파될 수 있다.

리프 노드 분할
↓
부모에 키 삽입
↓
부모도 가득 참
↓
부모 노드 분할
↓
필요하면 새로운 루트 생성

루트가 분할될 때만 트리의 높이가 1 증가한다.

[6] 삭제

키를 삭제했을 때 노드의 키가 최소 개수보다 작아질 수 있다. 이를 Underflow라고 한다.

키 삭제
↓
노드의 최소 키 개수 미달
↓
형제 노드에서 키를 빌림
또는
형제 노드와 병합

- 형제에게 빌리기
 - 형제 노드에 여유가 있으면 부모의 키를 이용해 키를 재배치한다.
- 형제와 병합하기
 - 형제에게 빌릴 수 없다면 부모의 키 하나와 형제 노드를 합친다.
 - 병합으로 인해 부모도 최소 키 개수보다 작아지면 위쪽 노드에서도 재조정이 발생한다.

[7] 디스크에서 B-Tree를 사용하는 이유

DB나 파일 시스템에서는 노드 하나를 디스크의 페이지 또는 블록 크기에 맞춰 구성할 수 있다.

디스크 페이지 하나

| 키 여러 개 + 자식 페이지 번호 |

한 번의 디스크 읽기로 여러 키를 가져올 수 있다.

예를 들어, 노드 하나가 자식 500개를 가질 수 있다면:

높이 1 → 약 500개 범위

높이 2 → 약 250,000개 범위

높이 3 → 약 125,000,000개 범위

매우 많은 데이터를 저장해도 트리 높이가 낮게 유지된다.

트리 높이가 낮음

↓

탐색하면서 읽는 노드 수가 적음

↓

디스크 I/O 감소

CPU에서 비교 연산을 몇 번 더 하는 것보다 느린 SSD/HDD 접근 횟수를 줄이는 것이 훨씬 중요하다.